UNIT 6 70 min

Functions

Define reusable code with def, pass inputs, return results, and understand local versus global scope.

By the end of this unit you will be able to

- Define a function with `def` and call it
- Distinguish parameters from arguments
- Return a value with `return` and explain what happens when there is no `return`
- Use default and keyword arguments
- Explain why variables inside a function are local

Why functions

A **function** is a named block of code you can run as many times as you need. Functions make programs shorter, easier to test, and easier to change, because the logic lives in one place. You have been calling functions since Unit 1: `print()`, `range()`, and `int()` are all functions.

Defining and calling

greet.py

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ada")
greet("Ben")
```

- `def` starts the definition. The name follows, then parentheses, then a colon.
- **Parameters** are the names listed in the definition (`name`).
- **Arguments** are the values you pass when calling (`"Ada"`).

Returning values

`return` sends a value back to the caller and ends the function. A function without a `return` gives back `None`, Python's "no value" marker. Use `return` when the caller needs the result; use `print()` when the goal is only to show something.

Trace: a function with a return value TRACE

```
1  def add_tax(price, rate=0.07):
2      tax = price * rate
3      return price + tax
4
5  total = add_tax(100)
6  print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`add_tax = function`	—
2	2	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
3	3	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
4	5	`add_tax = function, total = 107.0`	—
5	6	`add_tax = function, total = 107.0`	`107.0`

Default and keyword arguments

A parameter can have a default value. Callers may leave it out. You can also pass arguments by name, which makes calls easier to read.

defaults.py

```
def greet(name="friend", punctuation="!"):
    print(f"Hi {name}{punctuation}")

greet()
greet("Maya")
greet(punctuation="?", name="Sam")
```

Scope: where variables live

A variable created inside a function is **local**. It exists only during that call. A variable created at the top level of a file is **global**. A function can *read* a global, but assigning to a name inside a function creates a new local variable instead of changing the global one.

scope.py

```
total = 10

def try_to_change():
    total = 99          # creates a new local variable
    print("inside:", total)

try_to_change()
print("outside:", total)
```

> **Watch out**
> Using a local variable outside its function raises `NameError`. Return the value instead of relying on a shared variable.

Good function habits

- Give each function one clear job and a verb name: `calculate_tax`, not `stuff`.
- Add a short docstring, a string as the first line of the body, that says what the function does.
- Prefer passing values in and returning results over changing globals.

docstring.py

```
def is_even(n):
    """Return True if n is divisible by 2."""
    return n % 2 == 0

print(is_even(4))
print(is_even(7))
```

Predict the output THINK FIRST

Remember: a missing `return` gives `None`.

```
def double(x):
    return x * 2

print(double(4))
```

Answer: `8`

```
def hi(name="friend"):
    print("Hi", name)

hi()
```

Answer: `Hi friend`

```
n = 5
def bump(n):
    n = n + 1
    return n

bump(n)
print(n)
```

Answer: `5`
The function returned 6 but nothing kept it, and the global `n` was never changed.

```
def check(x):
    if x > 0:
        return "positive"

print(check(-1))
```

Answer: `None`
No `return` ran, so the function gives back `None`.

Practice

▼ **Exercise 1 — Celsius to Fahrenheit**

Write a function `c_to_f(celsius)` that returns the Fahrenheit value (`celsius * 9 / 5 + 32`). Print the result for 100 and for 0.

▼ **Show solution**

Solution

```
def c_to_f(celsius):
    return celsius * 9 / 5 + 32

print(c_to_f(100))
print(c_to_f(0))
```

▼ **Exercise 2 — Largest of three**

Write `largest(a, b, c)` that returns the biggest of the three numbers **without** using the built-in `max()`. Test it with 4, 9, and 2.

▼ **Hint**

Start by assuming `a` is the biggest, then compare it to `b` and `c` with `if` statements.

▼ **Show solution**

Solution

```
def largest(a, b, c):
    best = a
    if b > best:
        best = b
    if c > best:
        best = c
    return best

print(largest(4, 9, 2))
```

Discussion

1. Why does `scope.py` print 99 inside and 10 outside? What would make a beginner expect the outside value to change?
2. When would you want a default argument, and when would you rather require the caller to supply the value?
3. If you wrote one function that does three jobs, how would you split it?

▼ **Facilitator notes**

Ask a student to explain the difference between `print()` and `return` in their own words. Most confusion in this unit is between the two.

Key terms

Function

A named, reusable block of code, defined with `def` and run by calling it.

Parameter

A variable name in a function definition that receives a value when the function is called.

Argument

The actual value passed to a function when it is called.

return

A statement that ends a function and sends a value back to the caller.

None

Python's special value meaning "no value". Functions without a `return` produce it.

Default argument

A parameter with a preset value, such as `rate=0.07`, used when the caller does not supply one.

Keyword argument

An argument passed by name, such as `greet(name="Sam")`.

Local variable

A variable that exists only inside the function where it was created.

Global variable

A variable defined at the top level of a program, visible to the code that follows it.

Docstring

A string literal placed as the first line of a function body to document what it does.

See all terms in the glossary →